

Exploratory Data Analysis - Laptops Pricing dataset

0.1 Objectives

- Visualize individual feature patterns
- Run descriptive statistical analysis on the dataset
- Use groups and pivot tables to find the effect of categorical variables on price
- Use Pearson Correlation to measure the interdependence between variables

Importing Required Libraries

```
[9]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
%matplotlib inline
```

0.2 Import the dataset

```
[10]: file_path="https://cf-courses-data.s3.us.cloud-object-storage.appdomain.
→cloud/IBMDeveloperSkillsNetwork-DA0101EN-Coursera/
→laptop_pricing_dataset_mod2.csv"
```

```
[11]: df = pd.read_csv(file_path, header=0)
```

Print the first 5 entries of the dataset to confirm loading.

```
[13]: df.head()
```

```
[13]:   Unnamed: 0.1  Unnamed: 0  Manufacturer  Category  GPU  OS  CPU_core  \
0              0              0         Acer         4    2    1           5
1              1              1         Dell         3    1    1           3
```

2

2	2	2	Dell	3	1	1	7
3	3	3	Dell	4	2	1	5
4	4	4	HP	4	2	1	7

	Screen_Size_inch	CPU_frequency	RAM_GB	Storage_GB_SSD	Weight_pounds
→ \					
0	14.0	0.551724	8	256	3.52800
1	15.6	0.689655	4	256	4.85100
2	15.6	0.931034	8	256	4.85100
3	13.3	0.551724	8	128	2.69010
4	15.6	0.620690	8	256	4.21155

	Price	Price-binned	Screen-Full_HD	Screen-IPS_panel
0	978	Low	0	1
1	634	Low	1	0
2	946	Low	1	0
3	1244	Low	0	1
4	837	Low	1	0

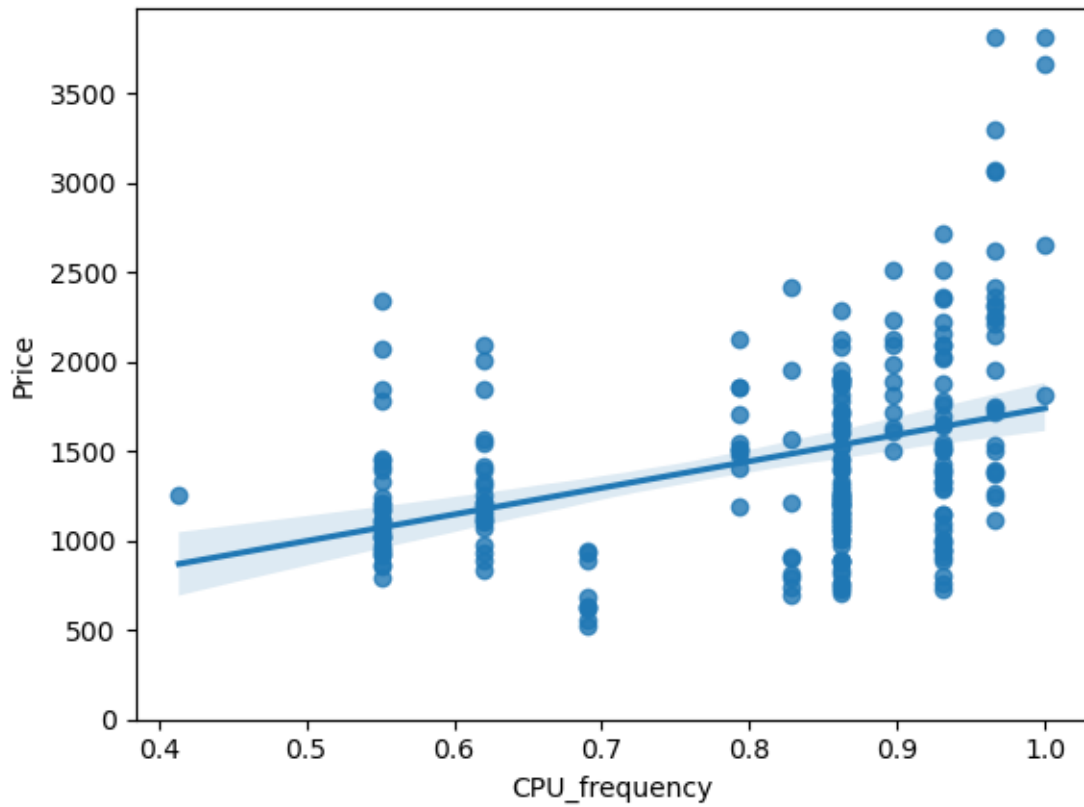
0.3 Visualize individual feature patterns

Continuous valued features

Let's generate regression plots for each of the parameters "CPU_frequency", "Screen_Size_inch" and "Weight_pounds" against "Price", and, print the value of correlation of each feature with "Price".

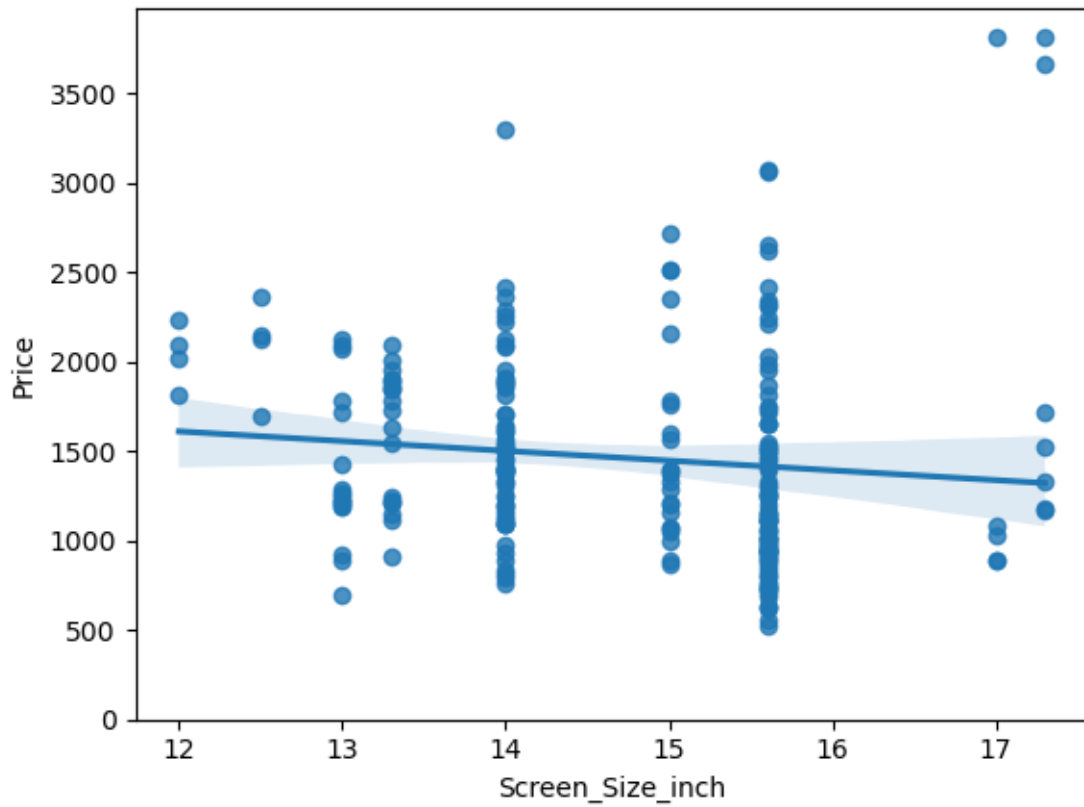
```
[14]: # CPU_frequency plot
sns.regplot(x="CPU_frequency", y="Price", data=df)
plt.ylim(0,)
```

```
[14]: (0.0, 3974.15)
```



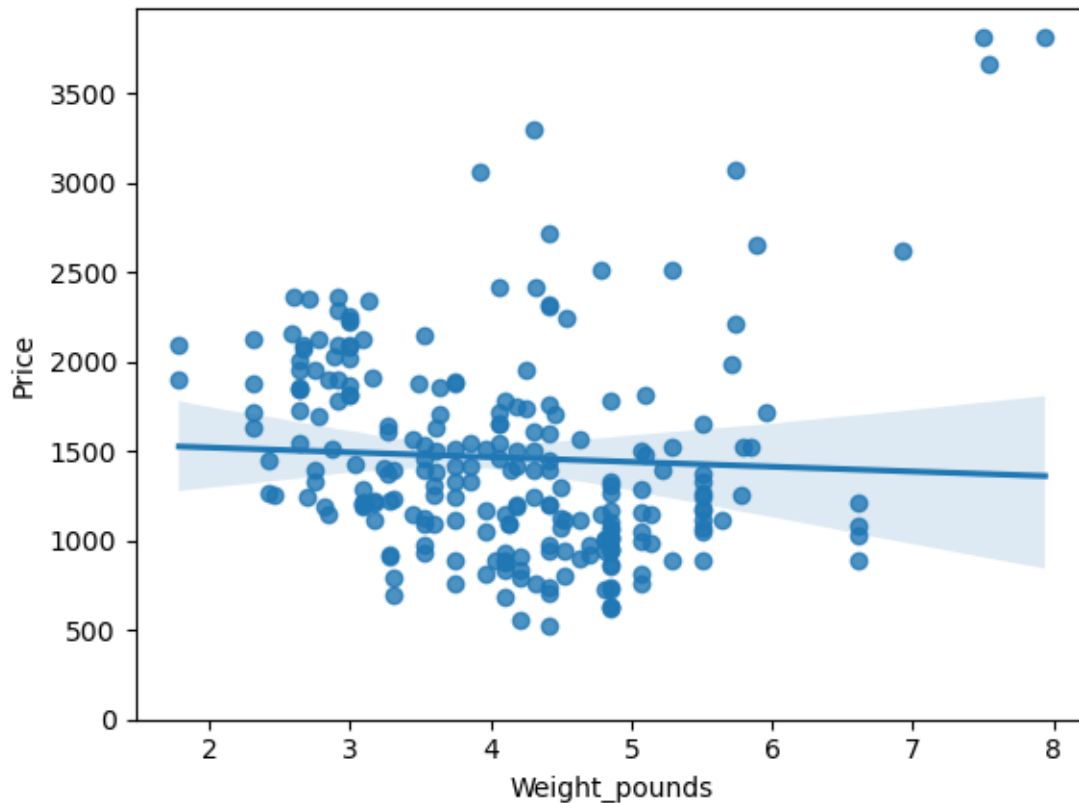
```
[17]: # Screen_Size_inch plot
sns.regplot(x="Screen_Size_inch" , y="Price", data=df)
plt.ylim(0,)
```

```
[17]: (0.0, 3974.15)
```



```
[18]: # Weight_pounds plot
sns.regplot(x="Weight_pounds" , y="Price", data=df)
plt.ylim(0,)
```

```
[18]: (0.0, 3974.15)
```



```
[19]: # Correlation values of the three attributes with Price
df[["CPU_frequency", "Screen_Size_inch", "Weight_pounds", "Price"]].corr()
```

```
[19]:
```

	CPU_frequency	Screen_Size_inch	Weight_pounds	Price
CPU_frequency	1.000000	-0.000948	0.066522	0.366666
Screen_Size_inch	-0.000948	1.000000	0.797534	-0.110644
Weight_pounds	0.066522	0.797534	1.000000	-0.050312
Price	0.366666	-0.110644	-0.050312	1.000000

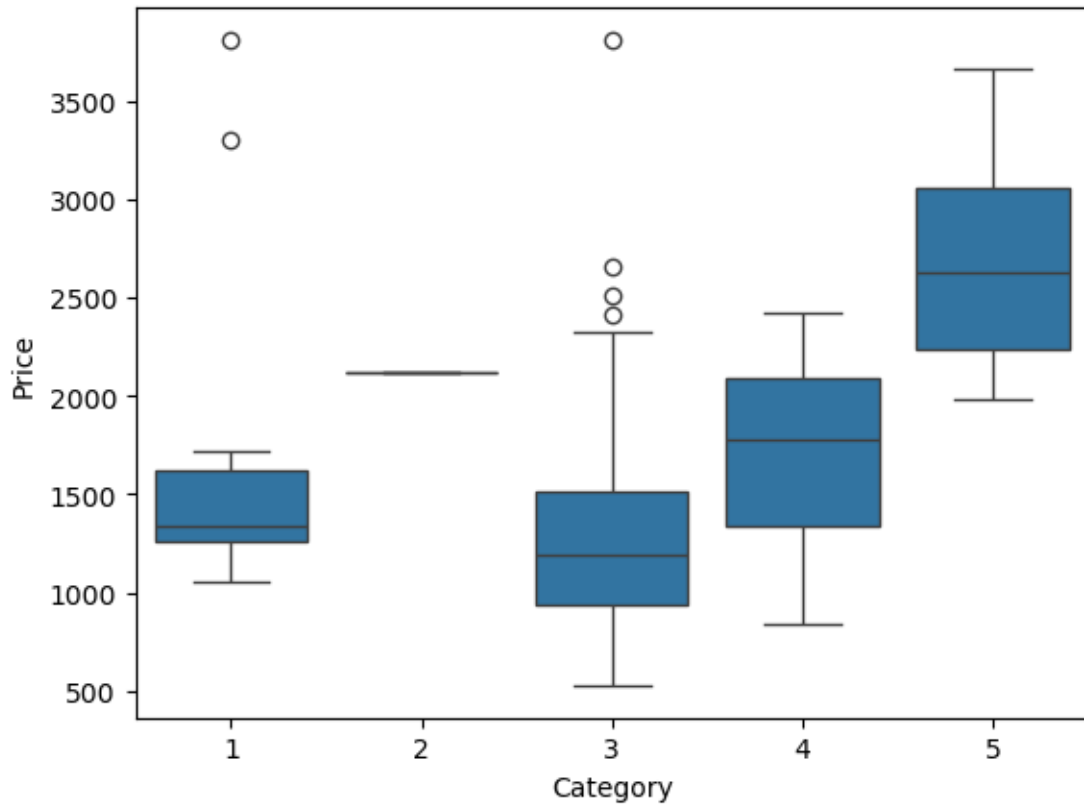
Interpretation: “CPU_frequency” has a 36% positive correlation with the price of the laptops. The other two parameters have weak correlation with price.

Categorical features

Let’s generate Box plots for the different feature that hold categorical values. These features would be “Category”, “GPU”, “OS”, “CPU_core”, “RAM_GB”, “Storage_GB_SSD”

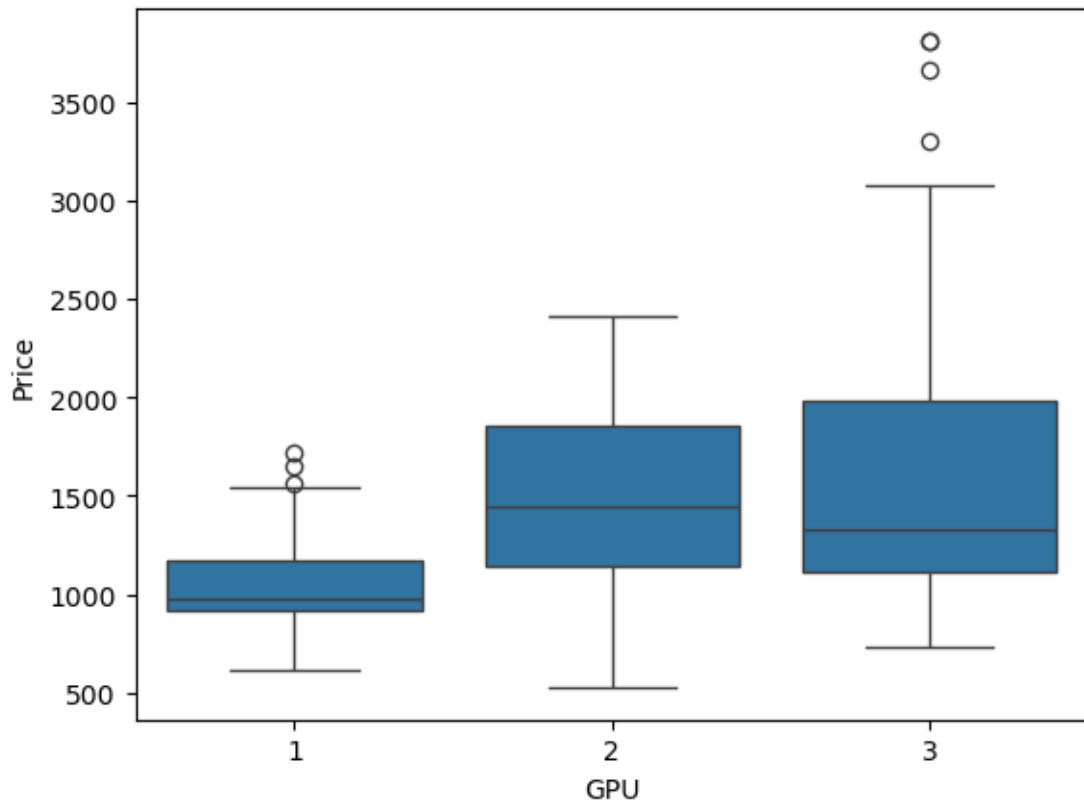
```
[21]: # Category Box plot
sns.boxplot(x="Category", y="Price", data=df)
```

```
[21]: <Axes: xlabel='Category', ylabel='Price'>
```



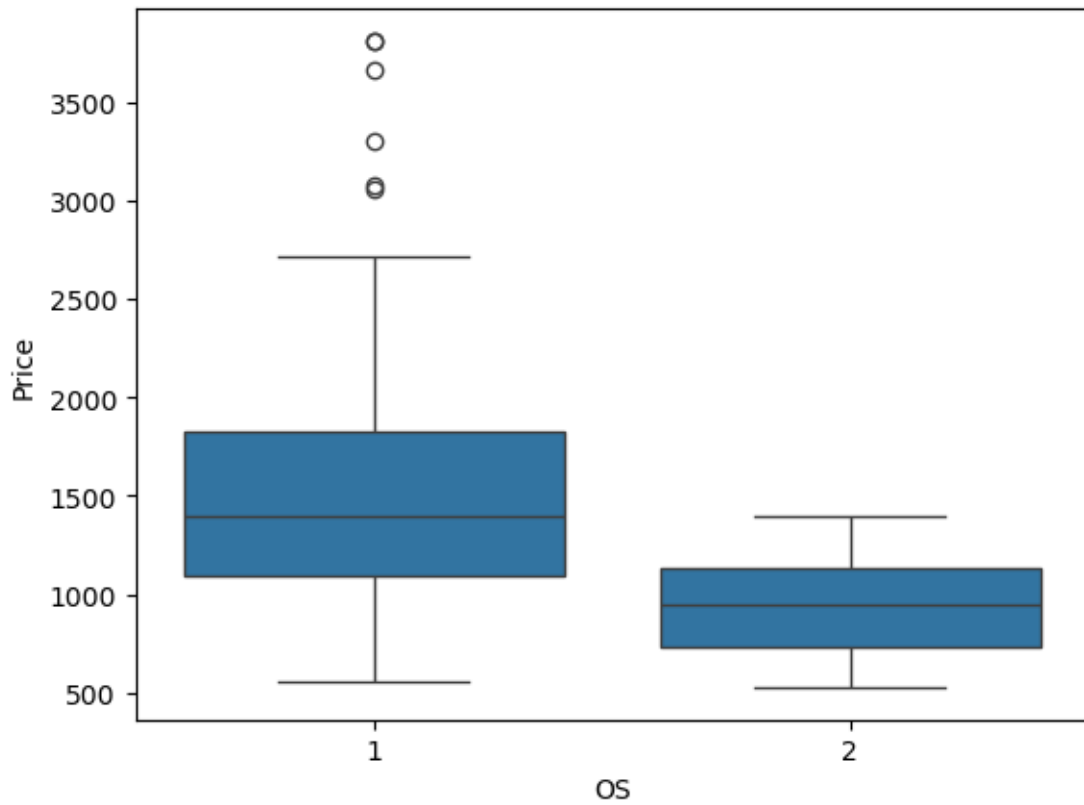
```
[22]: # GPU Box plot
sns.boxplot(x="GPU", y="Price", data=df)
```

```
[22]: <Axes: xlabel='GPU', ylabel='Price'>
```



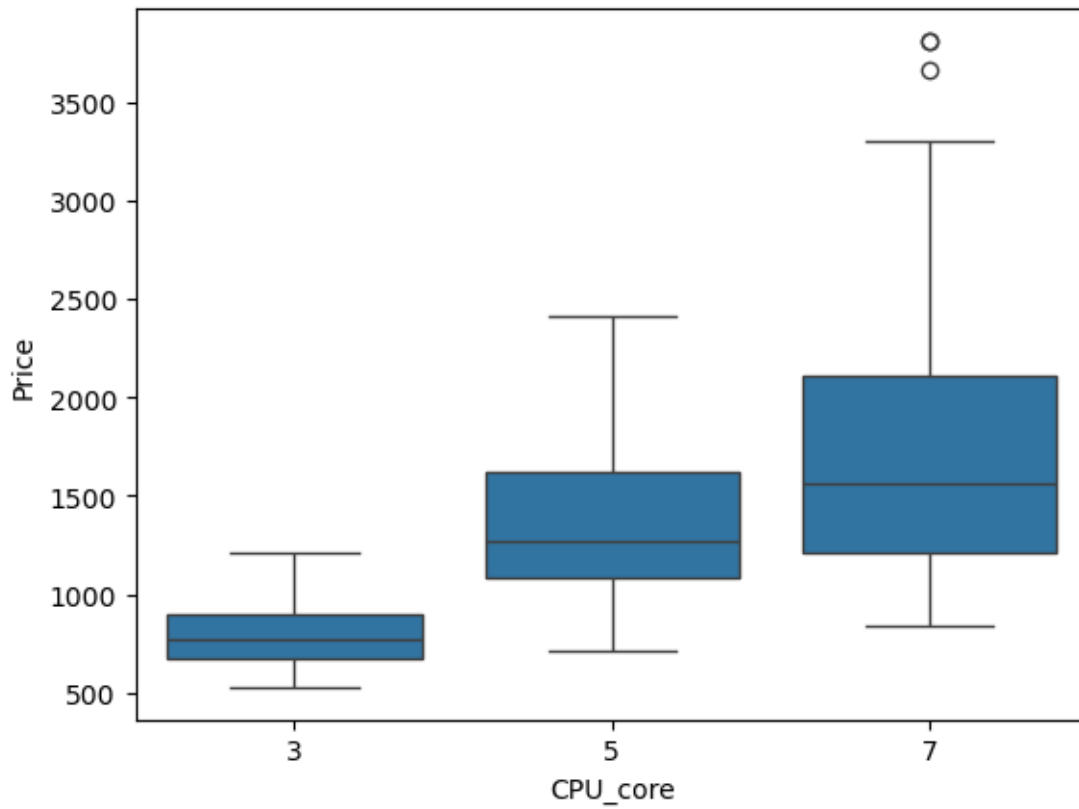
```
[23]: # OS Box plot
sns.boxplot(x="OS", y="Price", data=df)
```

```
[23]: <Axes: xlabel='OS', ylabel='Price'>
```



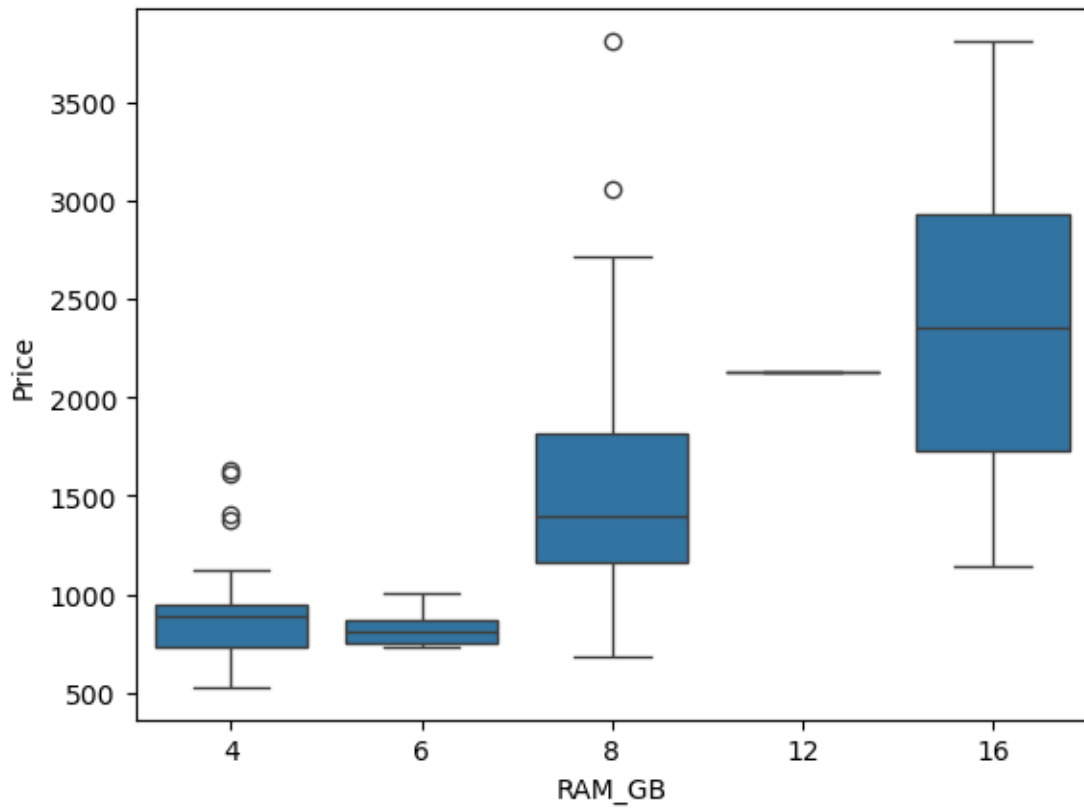
```
[24]: # CPU_core Box plot  
sns.boxplot(x="CPU_core", y="Price", data=df)
```

```
[24]: <Axes: xlabel='CPU_core', ylabel='Price'>
```

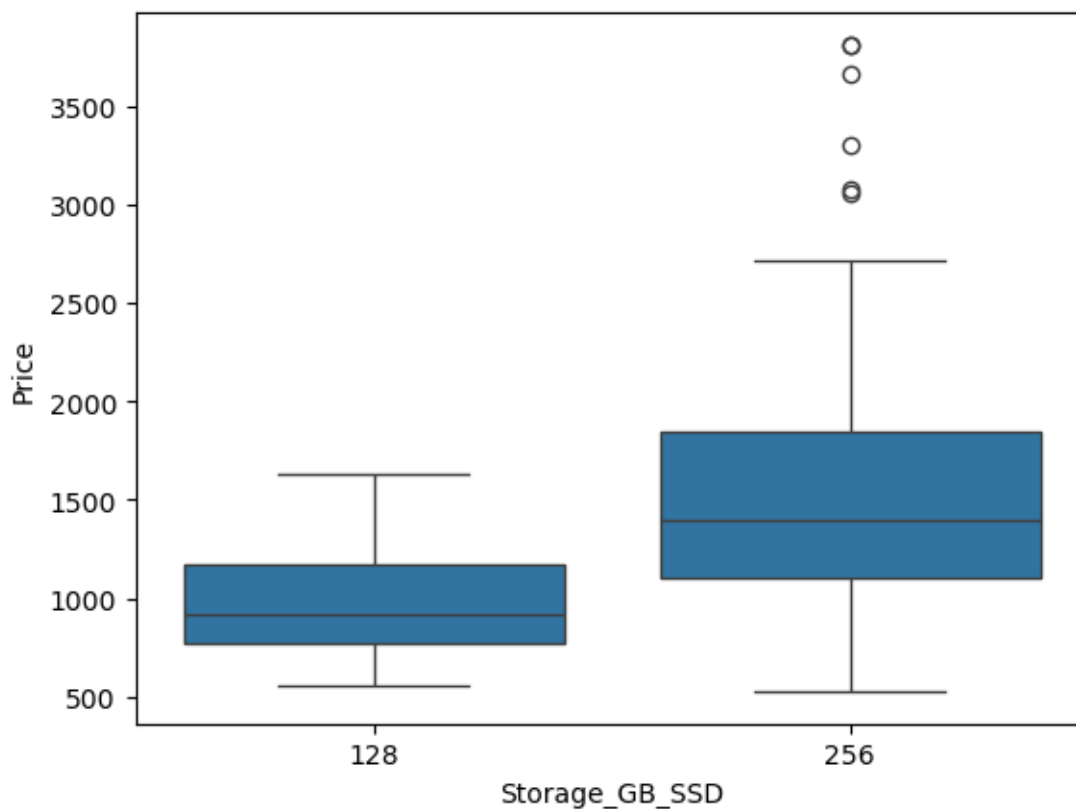
```
[25]: # RAM_GB Box plot
sns.boxplot(x="RAM_GB", y="Price", data=df)
```

```
[25]: <Axes: xlabel='RAM_GB', ylabel='Price'>
```



```
[26]: # Storage_GB_SSD Box plot
sns.boxplot(x="Storage_GB_SSD", y="Price", data=df)
```

```
[26]: <Axes: xlabel='Storage_GB_SSD', ylabel='Price'>
```



0.4 Descriptive Statistical Analysis

let's generate the statistical description of all the features being used in the data set.

```
[27]: df.describe()
```

```
[27]:
```

	Unnamed: 0.1	Unnamed: 0	Category	GPU	OS \
count	238.000000	238.000000	238.000000	238.000000	238.000000
mean	118.500000	118.500000	3.205882	2.151261	1.058824
std	68.848868	68.848868	0.776533	0.638282	0.235790
min	0.000000	0.000000	1.000000	1.000000	1.000000
25%	59.250000	59.250000	3.000000	2.000000	1.000000
50%	118.500000	118.500000	3.000000	2.000000	1.000000
75%	177.750000	177.750000	4.000000	3.000000	1.000000
max	237.000000	237.000000	5.000000	3.000000	2.000000

	CPU_core	Screen_Size_inch	CPU_frequency	RAM_GB \
count	238.000000	238.000000	238.000000	238.000000
mean	5.630252	14.688655	0.813822	7.882353

std	1.241787	1.166045	0.141860	2.482603
min	3.000000	12.000000	0.413793	4.000000
25%	5.000000	14.000000	0.689655	8.000000
50%	5.000000	15.000000	0.862069	8.000000
75%	7.000000	15.600000	0.931034	8.000000
max	7.000000	17.300000	1.000000	16.000000

	Storage_GB_SSD	Weight_pounds	Price	Screen-Full_HD \
count	238.000000	238.000000	238.000000	238.000000
mean	245.781513	4.106221	1462.344538	0.676471
std	34.765316	1.078442	574.607699	0.468809
min	128.000000	1.786050	527.000000	0.000000
25%	256.000000	3.246863	1066.500000	0.000000
50%	256.000000	4.106221	1333.000000	1.000000
75%	256.000000	4.851000	1777.000000	1.000000
max	256.000000	7.938000	3810.000000	1.000000

	Screen-IPS_panel
count	238.000000
mean	0.323529
std	0.468809
min	0.000000
25%	0.000000
50%	0.000000
75%	1.000000
max	1.000000

0.5 GroupBy and Pivot Tables

Let's group the parameters "GPU", "CPU_core" and "Price" to make a pivot table and visualize this connection using the pcolor plot.

```
[28]: # Create the group
df_group = df[["GPU", "CPU_core", "Price"]]
grouped = df_group.groupby(["GPU", "CPU_core"], as_index=False).mean()
grouped
```

```
[28]:   GPU  CPU_core  Price
0    1         3  769.250000
1    1         5  998.500000
2    1         7 1167.941176
3    2         3  785.076923
4    2         5 1462.197674
5    2         7 1744.621622
```

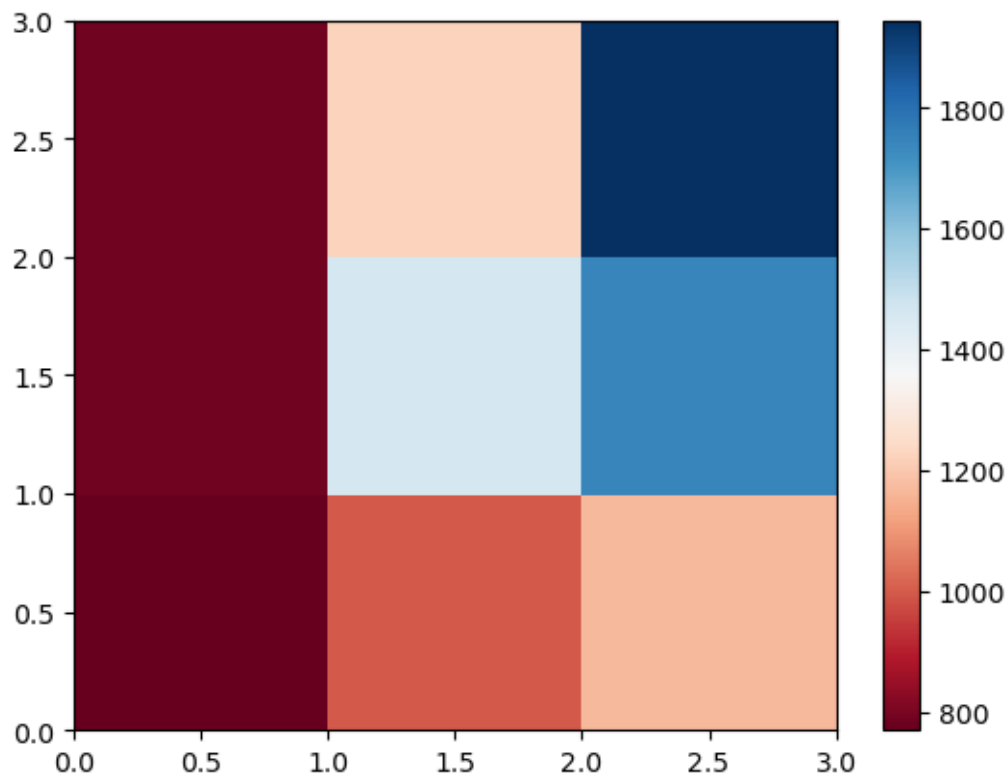
6	3	3	784.000000
7	3	5	1220.680000
8	3	7	1945.097561

```
[29]: # Create the Pivot table
group_pivot = grouped.pivot(index='GPU', columns='CPU_core')
group_pivot
```

```
[29]:
```

	Price		
CPU_core	3	5	7
GPU			
1	769.250000	998.500000	1167.941176
2	785.076923	1462.197674	1744.621622
3	784.000000	1220.680000	1945.097561

```
[30]: # Create the Plot
plt.pcolor(group_pivot, cmap='RdBu')
plt.colorbar()
plt.show()
```



0.6 Pearson Correlation and p-values

Let's use the `scipy.stats.pearsonr()` function to evaluate the Pearson Coefficient and the p-values for each parameter tested above. This will help you determine the parameters most likely to have a strong effect on the price of the laptops.

```
[31]: for i in_
    → ['RAM_GB', 'CPU_frequency', 'Storage_GB_SSD', 'Screen_Size_inch', 'Weight_pounds', 'CPU_core']
    →
        pearson_coef, p_value = stats.pearsonr(df[i], df["Price"])
        print(i)
        print("pearson coef is", pearson_coef, "with p_value", p_value)
```

```
RAM_GB
pearson coef is 0.5492972971857841 with p_value 3.681560628842995e-20
CPU_frequency
pearson coef is 0.36666555892588604 with p_value 5.502463350713357e-09
Storage_GB_SSD
pearson coef is 0.24342075521810297 with p_value 0.0001489892319172414
Screen_Size_inch
pearson coef is -0.11064420817118273 with p_value 0.08853397846830661
Weight_pounds
pearson coef is -0.050312258377515524 with p_value 0.4397693853433896
CPU_core
pearson coef is 0.4593977773355115 with p_value 7.912950127009183e-14
OS
pearson coef is -0.22172980114827384 with p_value 0.0005696642559246719
GPU
pearson coef is 0.2882981988881427 with p_value 6.16694969836445e-06
Category
pearson coef is 0.28624275581264125 with p_value 7.22569623580658e-06
```

```
[ ]:
```